

Lezione 12: Classi Astratte, Interfacce ed Ereditarietà Multipla

Questo notebook raccoglie tutto il codice della Lezione 12 (MavenDate):

- `src/main/java/it/oop/core/Time.java` (Interfaccia)
- `src/main/java/it/oop/core/Date.java` (Formattazione compatta `y%dm%dd%d`)
- `src/main/java/it/oop/core/FormattedDate.java` (Classe astratta)
- `src/main/java/it/oop/core/ItalianDate.java`
- `src/main/java/it/oop/core/AmericanDate.java`
- `src/main/java/it/oop/core/TimeStamp.java` (Implementa interfaccia `Time` ed estende `Date`)
- `src/main/java/it/oop/core/BirthDay.java`
- `src/main/java/it/oop/ui/MainDate.java`
- `src/test/java/it/oop/core/TestItalianDate.java`

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/
├── codice-commentato/
│   └── Lezione12/
│       ├── MavenDate
│       │   ├── pom.xml
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   └── it
│       │       │   │       └── oop
│       │       │   │           ├── core
│       │       │   │           │   ├── AmericanDate.java
│       │       │   │           │   ├── BirthDay.java
│       │       │   │           │   ├── Date.java
│       │       │   │           │   ├── FormattedDate.java
│       │       │   │           │   ├── ItalianDate.java
│       │       │   │           │   ├── Time.java
│       │       │   │           │   └── TimeStamp.java
│       │       │   │           └── ui
│       │       │   │               ├── Date.java
│       │       │   │               └── MainDate.java
│       │       │   └── resources
│       │       └── test
│       │           ├── java
│       │           │   └── it
│       │           │       └── oop
│       │           │           └── core
│       │           │               └── TestItalianDate.java
```

Argomenti trattati:

- Classi astratte con la parola chiave `abstract` (`FormattedDate`)
- Definizione di interfacce con `interface` (`Time`) con metodi implicitamente `public abstract`
- Ereditarietà multipla di tipo: una classe estende una superclasse e implementa un'interfaccia (`TimeStamp extends Date implements Time`)
- Metodi finali (`final String printFormat()`) per prevenire l'overriding incontrollato
- Polimorfismo con collezioni generiche (`List<Date>`)

1. Interfaccia `Time`

```
In [1]: interface Time {
        int getSeconds();
        int getMinutes();
```

```
    int getHours();  
}
```

2. Classe `Date`

```
In [2]: class Date {  
    protected int day;  
    protected int month;  
    protected int year;  
  
    public Date(int day, int month, int year) {  
        this.day = day;  
        this.month = month;  
        this.year = year;  
        verify();  
    }  
    public Date(int day, int month) {  
        this(day, month, 2025);  
    }  
    public Date(Date other) {  
        this.day = other.day;  
        this.month = other.month;  
        this.year = other.year;  
        verify();  
    }  
  
    void verify() {  
        if (year < 0 || month < 1 || month > 12)  
            System.out.println("Illegal date!"); // Illegal date!  
        else  
            if (day < 1 || day > daysPerMonth(month))  
                System.out.println("Illegal date!"); // Illegal date!  
    }  
  
    public int getDay() { return day; }  
    public int getMonth() { return month; }  
    public int getYear() { return year; }  
  
    public static int daysPerMonth(int month) {  
        int days;  
        switch(month) {  
            case 4:  
            case 6:  
            case 9:  
            case 11:  
                days = 30;  
                break;  
            case 2:  
                days = 28;  
                break;  
            default:  
                days = 31;  
                break;  
        }  
        return days;  
    }  
  
    @Override  
    public String toString() {  
        return String.format("y%dm%dd%d", year, month, day);  
    }  
  
    @Override  
    public boolean equals(Object other) {  
        if (other == null) return false;  
        if (this == other) return true;  
        if (!(other instanceof Date)) return false;  
        Date otherAsDate = (Date) other;  
        return this.day == otherAsDate.getDay() &&  
            this.month == otherAsDate.getMonth() &&  
            this.year == otherAsDate.getYear();  
    }  
}
```

```
}  
}
```

3. Classe Astratta `FormattedDate` e Interfaccia `FormattedDateConverter`

```
In [3]: abstract class FormattedDate extends Date {  
    protected final String format;  
    protected final String[] months;  
  
    public FormattedDate(int day, int month, int year, String format, String[] months) {  
        super(day, month, year);  
        this.format = format;  
        this.months = months;  
    }  
  
    public final String printFormat() {  
        return format;  
    }  
  
    public final String getMonthAsString() {  
        return months[getMonth()-1];  
    }  
  
    public abstract String prettyPrint();  
}  
  
@FunctionalInterface  
interface FormattedDateConverter {  
    FormattedDate convert(FormattedDate date);  
}
```

4. Classe `ItalianDate`

```
In [4]: class ItalianDate extends FormattedDate {  
    private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",  
    "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };  
  
    public ItalianDate(int day, int month, int year) {  
        super(day, month, year, "dd/mm/yyyy", MONTHS_IT);  
    }  
  
    @Override  
    public String prettyPrint() {  
        return day + " " + getMonthAsString() + " " + getYear();  
    }  
  
    @Override  
    public String toString() {  
        return day + "/" + getMonth() + "/" + getYear();  
    }  
}
```

5. Classe `AmericanDate`

```
In [5]: class AmericanDate extends FormattedDate {  
    private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",  
    "July", "August", "September", "October", "November", "December" };  
  
    public AmericanDate(int day, int month, int year) {  
        super(day, month, year, "mm/dd/yyyy", MONTHS_US);  
    }  
  
    @Override  
    public String prettyPrint() {  
        return getMonthAsString() + " " + day + ", " + getYear();  
    }  
  
    @Override  
    public String toString() {
```

```

        return getMonth() + "/" + getDay() + "/" + getYear();
    }
}

```

6. Classe `TimeStamp` (Implementa `Time` ed estende `Date`)

```

In [6]: class TimeStamp extends Date implements Time {
    protected final int seconds;
    protected final int minutes;
    protected final int hours;

    public TimeStamp(int seconds, int minutes, int hours, int day, int month, int year) {
        super(day, month, year);
        this.seconds = seconds;
        this.minutes = minutes;
        this.hours = hours;
    }

    @Override
    public int getSeconds() {
        return seconds;
    }

    @Override
    public int getMinutes() {
        return minutes;
    }

    @Override
    public int getHours() {
        return hours;
    }

    @Override
    public String toString() {
        return String.format("%s[%02d:%02d:%02d]", super.toString(), hours, minutes, seconds);
    }

    @Override
    public boolean equals(Object other) {
        if (super.equals(other) == false) return false;
        if (!(other instanceof TimeStamp)) return false;
        TimeStamp otherAsTimeStamp = (TimeStamp) other;
        return hours == otherAsTimeStamp.getHours() &&
            minutes == otherAsTimeStamp.getMinutes() &&
            seconds == otherAsTimeStamp.getSeconds();
    }
}

```

7. Classe `TestItalianDate` ed Esecuzione

```

In [7]: class TestItalianDate {
    private static ItalianDate date = new ItalianDate(1, 1, 1970);

    private static void printFormatTest() {
        assert date.printFormat().equals("dd/mm/yyyy");
    }

    public static void main(String[] args) {
        printFormatTest();
    }
}

TestItalianDate.main(null);

```

8. Classe `MainDate` ed Esecuzione

```

In [8]: import java.util.List;

class MainDate {
    public static void main(String[] args) {
        Date date = new Date(3, 11, 2025);
    }
}

```

```

        ItalianDate itDate = new ItalianDate(3, 11, 2025);
        AmericanDate usDate = new AmericanDate(3, 11, 2025);
        System.out.println("date: " + date.toString()); // date: y2025m11d3
        System.out.println("itDate: " + itDate.toString()); // itDate: 3/11/2025
        System.out.println("usDate: " + usDate.toString()); // usDate: 11/3/2025
        System.out.println("itDate format: " + itDate.printFormat()); // itDate format: dd/mm/yyyy
        System.out.println("usDate format: " + usDate.printFormat()); // usDate format: mm/dd/yyyy
        System.out.println("date ?= itDate: " + date.equals(itDate)); // date ?= itDate: true
        System.out.println("itDate ?= usDate: " + itDate.equals(usDate)); // itDate ?= usDate: true
        System.out.println("date ?= 4/11/2025: " + date.equals(new ItalianDate(4, 11, 2025))); // date ?=
4/11/2025: false
        FormattedDate id = new ItalianDate(3, 11, 2025);
        System.out.println("id format: " + id.printFormat()); // id format: dd/mm/yyyy
        Time ts = new TimeStamp(10, 11, 2025, 10, 0, 0);
        System.out.println(ts.toString()); // y2025m11d10[10:00:00]
        System.out.println(ts.getHours()); // 10
        System.out.println(((TimeStamp) ts).getDay()); // 10
        List<Date> dates = List.of(new Date(9,9,2025), (Date) id, (Date) ts);
        System.out.println(dates.toString()); // [y2025m9d9, 3/11/2025, y2025m11d10[10:00:00]]
    }
}
MainDate.main(null);

```

```

date: y2025m11d3
itDate: 3/11/2025
usDate: 11/3/2025
itDate format: dd/mm/yyyy
usDate format: mm/dd/yyyy
date ?= itDate: true
itDate ?= usDate: true
date ?= 4/11/2025: false
id format: dd/mm/yyyy
Illegal date!
y0m0d10[2025:11:10]
2025
10
[y2025m9d9, 3/11/2025, y0m0d10[2025:11:10]]

```